

Stored Procedures and Functions in SQL: One Fix, Live Everywhere

8. Introduction

Every backend engineer eventually hits the same wall: the *same* piece of SQL logic scattered across five different applications. A reporting service has it. The mobile API has a copy. That old cron job has a slightly-wrong version of it. Then a business rule changes, and now you are hunting through five codebases to make one fix.

Stored procedures and functions exist to end that pain. They let you write a piece of database logic **once**, save it *inside* the database, and call it by name from anywhere. Fix the procedure, and every app that calls it is instantly corrected. No redeploys of five services. No copy-paste sprawl.

This topic shows up constantly in **coding interviews** and backend system-design discussions because it sits at the intersection of three things interviewers care about: **code reuse**, **database security**, and **performance**. Knowing when to reach for a procedure, when to reach for a function, and when to reach for neither is a genuine mark of an experienced **software engineer**. Understanding parameter direction (IN, OUT, INOUT) and the classic row-by-row performance trap separates people who *use* databases from people who *understand* them.

By the end of this article, you will know exactly which tool to pick, how to write both cleanly, and which traps quietly wreck performance in production.

9. Problem Overview (In Plain Words)

Let's frame the real problem these features solve.

You have SQL logic — maybe an `INSERT` that adds a user, or a calculation that applies a 10% tax — and you need to run it repeatedly, from multiple places, safely, and fast.

Naively, you copy that SQL into every application that needs it. This creates four problems:

1. **Duplication** — the same logic lives in many places.
2. **Drift** — copies fall out of sync as some get updated and others don't.
3. **Network chatter** — ten separate statements mean ten round-trips to the database.

4. **Security exposure** — every app needs direct access to the underlying tables.

A **stored procedure** is a named block of SQL statements saved in the database. You call it by name, hand it some values, and it runs its saved job — inserting, updating, deleting, or several of those in one shot.

A **function** is a named block that takes input and **returns a single value** you can use directly inside a query — in a `SELECT` list, a `WHERE` clause, anywhere an expression is allowed.

The mental model: - **Procedure** = a **worker** who goes and *does tasks*. - **Function** = a **calculator** that *answers one question*.

10. Examples

Example A — A stored procedure that adds a user

```
CALL AddUser('Ada', 36);
```

Output: A new row (`'Ada'`, `36`) is inserted into the `users` table. Nothing is returned to you — the procedure *did an action*. You never retyped the `INSERT` ; you just handed over two values.

Example B — A function that computes tax

```
SELECT price, TaxOn(price) AS tax FROM products;
```

If a product's `price` is `200`, then `TaxOn(200)` returns `20`.

Output:

price	tax
200	20
50	5
999	99.9

The function slots into the `SELECT` as if it were built into SQL, returning one number per row.

Example C — An OUT parameter returning a value

```
CALL Doubler(5, @result);  
SELECT @result;    -- 10
```

Output: `10`. We pushed `5` in, and the `OUT` slot came back holding `10`.

11. Intuition

Here is how an experienced engineer arrives at these tools — without jumping straight to syntax.

Start with a frustration: *"I keep writing the same SQL."* When you write anything twice, your instinct should be to name it and reuse it. That instinct, applied to database logic, is exactly what a stored procedure is. Think of a **saved recipe** instead of shouting every ingredient each time you cook, or a **vending machine button** that triggers a whole chain of steps inside.

Now split that instinct in two, based on *what the reused logic does*:

- If the logic **performs actions** — insert, update, delete, maybe several — you want a **procedure**. It's a worker you dispatch.
- If the logic **computes and returns one answer** you want to use mid-query — you want a **function**. It's the **+** sign on a calculator: feed it numbers, get one back.

That single distinction — *does something* vs. *returns something you can plug into a query* — is the whole decision. Once you internalize it, parameters and security fall into place naturally.

Then layer on the three payoffs an experienced engineer immediately recognizes:

1. **Edit the master, not the photocopies.** Change one procedure instead of ten copies across five apps.
2. **One trip, not ten.** A single `CALL` crosses the network once instead of ten separate round-trips — turning a slow report from seconds into milliseconds.
3. **A controlled door, not an open vault.** Like a bank teller who fetches only what you're allowed to have, a procedure lets people run logic *without* direct access to the tables underneath.

12. Brute Force Solution (Inline SQL Everywhere)

Idea: Skip procedures and functions entirely. Write the raw SQL directly in each application, every time you need it.

Algorithm: 1. In each app, hardcode the SQL statement (e.g., the `INSERT`). 2. Send it over the network on every operation. 3. Grant each app direct table access.

Advantages: - Zero database-side setup. - Everything lives in application code, which many teams find easier to test and version.

Disadvantages: - Logic is duplicated across every app — copy-paste sprawl. - A rule change means editing and redeploying *every* app. - Chatty: many separate round-trips to the database. - Every app

needs broad table permissions, widening your attack surface.

Time Complexity: Per operation the SQL is the same, but **maintenance cost scales with the number of apps** — $O(\text{apps})$ for every logic change. **Space Complexity:** $O(\text{apps})$ duplicated logic across codebases.

This "works," but it's the version that pages you at 3am when copy #4 drifts out of sync.

13. Optimal Solution (Procedures + Functions)

Centralize the logic **once** inside the database.

Step 1 — Wrap the action in a procedure. Write the `INSERT` a single time, name it `AddUser`, and give it two IN parameters.

Step 2 — Wrap the computation in a function. Write the tax math once, name it `TaxOn`, and return one value.

Step 3 — Understand parameter direction. Parameters are labeled slots for passing values:

Direction	Meaning	Analogy
IN	Value goes into the procedure	A gift you give
OUT	Procedure sends a value back out	A gift you get
INOUT	Both — goes in, comes back changed	A gift you swap

Step 4 — Call by name from every app. Now all five apps just `CALL AddUser(...)` or use `TaxOn(...)`. One fix updates them all.

Step 5 — Lock the door. Grant apps permission to *run the procedure* without granting direct table access. Controlled access, smaller attack surface.

14. SQL Code

```
-- A stored procedure: performs an action, returns nothing directly.
DELIMITER //
CREATE PROCEDURE AddUser(IN user_name VARCHAR(100), IN user_age INT)
BEGIN
    INSERT INTO users (name, age)
    VALUES (user_name, user_age);
END //
DELIMITER ;

-- A function: computes and returns exactly one value.
```

```

DELIMITER //
CREATE FUNCTION TaxOn(price DECIMAL(10, 2))
RETURNS DECIMAL(10, 2)
DETERMINISTIC
BEGIN
    RETURN price * 0.10;    -- 10% tax
END //
DELIMITER ;

-- An OUT parameter: sends a value back out.
DELIMITER //
CREATE PROCEDURE Doubler(IN n INT, OUT result INT)
BEGIN
    SET result = n * 2;
END //
DELIMITER ;

```

Calling them:

```

CALL AddUser('Ada', 36);           -- action
SELECT price, TaxOn(price) FROM products;    -- computation inside a query
CALL Doubler(5, @out); SELECT @out;    -- 10 comes back

```

Python-friendly usage

Backend engineers rarely call SQL by hand — they invoke it from application code. Here's the same logic driven cleanly from **Python**:

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="app", password="secret", database="shop"
)
cursor = conn.cursor()

# Call the stored procedure by name – no raw INSERT in the app.
cursor.callproc("AddUser", ["Ada", 36])
conn.commit()

# Use the OUT parameter.
result_args = cursor.callproc("Doubler", [5, 0])
print(result_args[1])    # -> 10

# Use the function inside a normal query.
cursor.execute("SELECT price, TaxOn(price) FROM products")
for price, tax in cursor.fetchall():
    print(price, tax)

```

```
cursor.close()
conn.close()
```

15. Code Walkthrough

- `DELIMITER //` temporarily changes the statement terminator so the semicolons *inside* the procedure body don't end the definition early. We reset it with `DELIMITER ;` afterward.
- `CREATE PROCEDURE AddUser(IN user_name ..., IN user_age ...)` — both parameters are `IN` : values flow in, nothing flows back.
- The `BEGIN ... END` block holds the saved SQL — here, a single `INSERT`. It could hold many statements.
- `CREATE FUNCTION TaxOn(...) RETURNS DECIMAL` — a function **must** declare its return type and **must** `RETURN` a value.
- `DETERMINISTIC` tells the engine the same input always yields the same output, which helps it optimize.
- In `Doubler`, `OUT result INT` is the return channel; `SET result = n * 2` fills it.
- In Python, `cursor.callproc(name, args)` invokes a procedure by name; the returned list carries back any `OUT` / `INOUT` values.

16. Dry Run

Let's trace `CALL Doubler(5, @out)` end to end:

1. `n` receives `5` (an `IN` parameter).
2. `result` starts empty (an `OUT` parameter — a slot to be filled).
3. `SET result = n * 2` → `result = 5 * 2 = 10`.
4. The procedure ends; `10` is written back into `@out`.
5. `SELECT @out;` displays `10`.

We handed in one value and got one value back — a clean `IN` → `OUT` flow.

17. Complexity Analysis

- **Per-call runtime:** Whatever the SQL inside costs. A single `INSERT` is $O(1)$ plus index maintenance. The win isn't asymptotic — it's the **elimination of network round-trips**: one `CALL` instead of ten statements.

- **Maintenance complexity:** $O(1)$ to change a rule — you edit one procedure — versus **$O(\text{apps})$** with inline SQL. This is the real optimization.
- **Space:** The logic is stored once in the database rather than duplicated across every application. $O(1)$ versus $O(\text{apps})$.

Why these are correct: centralization removes duplicated work both at *change time* (one edit) and at *runtime* (one round-trip), which is precisely why a slow report can drop from seconds to milliseconds.

18. Alternative Solutions

Procedures aren't the only tool. Pick the **lightest** one that fits:

Approach	Best for	Trade-off
Application code	Logic you want to unit-test and version in Git	Easier to test, but chattier over the network
Stored procedure	Centralized, action-heavy, security-sensitive logic	Fast and centralized, but harder to debug and tied to your DB
View	A reusable <i>query</i> with no real logic	Simplest reuse; can't perform actions or take parameters

If all you need is a saved, reusable `SELECT` with no computation, a **view** is often all you need — don't reach for a procedure.

19. Edge Cases

- **Empty input / NULL parameters:** `AddUser(NULL, NULL)` will insert NULLs (or fail a NOT NULL constraint). Validate before inserting.
- **Duplicates:** Calling `AddUser` twice with the same data inserts two rows unless a UNIQUE constraint stops it.
- **Min/max values:** `TaxOn` on a huge `price` can overflow the declared `DECIMAL` precision — size the return type deliberately.
- **Zero and negatives:** `TaxOn(0)` returns `0`; `TaxOn(-100)` returns `-10`. Decide whether negative prices are even valid.
- **Uninitialized OUT parameter:** If a procedure forgets to `SET` its OUT parameter on some path, the caller gets a stale or NULL value.

20. Common Mistakes

1. **Putting a heavy function inside a `SELECT` over millions of rows.** The function runs once *per row*. A slow lookup buried in a function scanned over a million rows will crawl — this is a real, painful mistake. Keep functions **light and predictable**.
2. **The row-by-row loop (RBAR — "row by row agonizing row").** Writing a procedure that loops and updates one row at a time is like washing rice one grain at a time. Databases are built for **whole sets at once**. If you can replace a loop with a single `UPDATE`, do it.
3. **Burying all business rules in the database.** It becomes hard to test, hard to version, and hard for new engineers to find.
4. **Forgetting `DELIMITER`.** The inner semicolons end your definition early and throw confusing syntax errors.
5. **Confusing procedures and functions.** Trying to `SELECT AddUser(...)` fails — procedures don't return a value into a query. Trying to `INSERT` inside a function often isn't allowed either.
6. **Non-deterministic functions marked `DETERMINISTIC`.** Lying to the optimizer causes subtly wrong cached results.

21. Interview Questions (Likely Follow-Ups)

- What's the difference between a stored procedure and a function? When would you pick each?
- Explain IN, OUT, and INOUT parameters with an example.
- Why can calling a function inside a large `SELECT` destroy performance?
- What is the row-by-row (RBAR) anti-pattern, and how do you fix it?
- How do stored procedures improve **database security**?
- When would a simple *view* be a better choice than a procedure?
- What are the downsides of putting business logic in the database?

22. Similar Problems

- **LeetCode 177 – Nth Highest Salary:** Officially solved by writing a **SQL function** that returns one value — direct practice for function syntax and return semantics.
- **LeetCode 1873 – Calculate Special Bonus:** Uses `CASE` logic that's a natural fit for a reusable function.
- **LeetCode 176 – Second Highest Salary:** Reinforces returning a single scalar value (including handling `NULL`) — the exact contract a function must honor.
- **Any "create a view" problem:** Sharpens the instinct for choosing the *lightest* reuse tool, echoing our alternatives table.

These are related because they all force you to decide *what the reusable unit returns* and *which SQL construct fits* — the core lesson here.

23. Key Takeaways

- A **stored procedure** is a saved button that performs actions; a **function** is a calculator that returns one value you use inside queries.
- They cut repeated code, reduce network chatter, and guard your tables behind a controlled door.
- **IN** is a gift you give; **OUT** is a gift you get; **INOUT** does both.
- Keep functions **light**, avoid **row-by-row loops**, and don't hide your entire app inside the database.
- Always pick the **lightest tool** that fits: app code, procedure, or a plain view.

24. Watch the Video

Seeing these built live makes the IN/OUT/INOUT distinction click far faster than reading syntax. Watch the full walkthrough — we build a tiny procedure and a tiny function from scratch, break the parameters down, and demo the exact performance trap that bites beginners: 👉 **{LINK}**

25. About the Series

This is part of the **Fun with Learning Technology** series — a friendly, no-fluff journey through databases, algorithms, and backend engineering, one focused lesson at a time. Each episode takes a single practical concept, builds real intuition before any code, and shows the traps that only show up in production. If you're learning **SQL**, preparing for **coding interviews**, or leveling up as a **software engineer**, the series is built to grow with you, lesson by lesson.

26. Call To Action

If this made stored procedures and functions click, here's how to help the series grow:

- 👍 **Like** the video on YouTube — it tells the algorithm the lesson landed.
 - 🔔 **Subscribe** on YouTube and hit the bell — a follow means come back tomorrow.
 - 📧 **Subscribe on Substack** so each new lesson lands in your inbox.
 - 💬 **Comment** with the SQL topic you want next.
 - 🔗 **Share** this with a developer who keeps copy-pasting the same SQL.
-

Quick Review

Trigger: repeated multi-step SQL logic reused across many app clients?

Think stored procedure or function — a named, precompiled SQL block in the DB centralizing logic, cutting network round-trips, enforcing consistency.

Time complexity of a stored procedure?

Same as the SQL it runs; the procedure wrapper adds no asymptotic cost. A row-by-row cursor loop is $O(n)$ calls versus one set-based statement.

Space complexity of stored procedures/functions?

$O(1)$ stored definition plus whatever the query needs; cursors and OUT/temp result sets add space proportional to rows processed.

Common bug: procedural cursor loops instead of set operations?

Row-by-row (RBAR) processing is slow and non-atomic. Rewrite as a single set-based statement; reserve cursors for genuinely sequential logic.

Procedure vs function: how do they differ?

Functions return one value, are called inside queries, and avoid side effects. Procedures use IN/OUT/INOUT params, can modify data, and are CALL-ed standalone.

Follow-up: how do stored procedures improve security?

Grant EXECUTE without table access, so callers run vetted logic only. Parameterized inputs resist SQL injection and hide the schema.

sql Lesson 27: Stored Procedures and Functions (Stored procedures and functions are reusable blocks of SQL code stored within the database to perform specific operations efficiently. They provide a mechanism for modularizing business logic, which enhances security, reduces network traffic, and enforces consistency across multiple database clients. You should utilize these tools whenever you need to abstract complex queries, ensure atomic execution of multi-step processes, or maintain a centralized library of database operations.) — free Python cheat sheet